process in *Internet Information Services (*IIS). The main alternative, HTTP.sys, runs only in Windows and can't be used with IIS.[1]

Kestrel is responsible for receiving the request data and constructing a .NET representation of the request, but it doesn't attempt to generate a response directly. For that task, Kestrel hands the `HttpContext` to the middleware pipeline in every ASP.NET Core application. This pipeline is a series of components that process the incoming request to perform common operations such as logging, handling exceptions, and serving static files.

NOTE You'll learn about the middleware pipeline in detail in chapter 4.

At the end of the middleware pipeline is the *endpoint* middleware, which is responsible for calling the code that generates the final response. In most applications that code will be a Model-View-Controller (MVC), Razor Pages, or minimal API endpoint.

Most ASP.NET Core applications follow this basic architecture, and the example in this chapter is no different. First, you'll see how to create and run your application; then you'll look at how the code corresponds to the outline in figure 3.1. Without further ado, let's create an application!

3.2 Creating your first ASP.NET Core application

In this section you're going to create a minimal API application that returns `"Hello World!"` when you call the HTTP API. This application is about the simplest ASP.NET Core application you can create, but it demonstrates many of the fundamental concepts of building and running applications with .NET.

You can start building applications with ASP.NET Core in many ways, depending on the tools and operating system you're using. Each set of tools has slightly